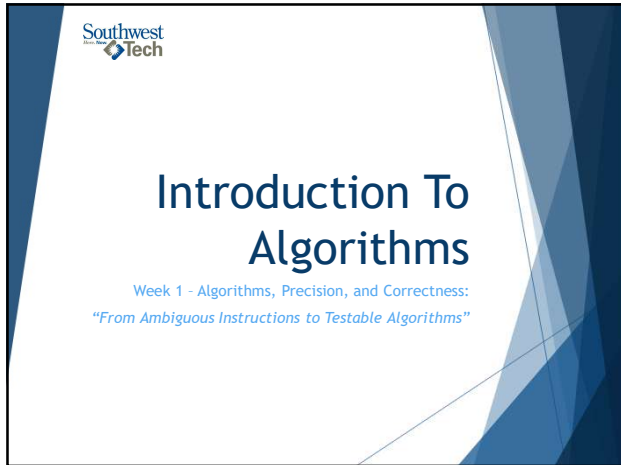
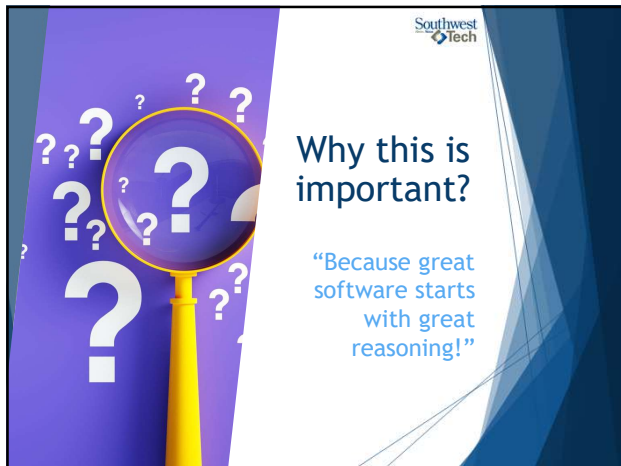
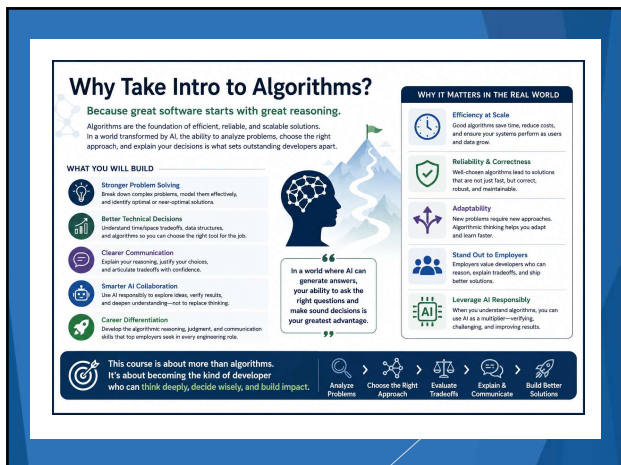




1



2



3

Southwest
Tech

Algorithmic Problem Solving

problem -> steps -> evidence -> revision.

-  - Algorithms are not just code.
-  - Algorithms are structured ways to solve problems.
-  - This course focuses on designing, testing, explaining, and improving them.

7

Terms to Remember

Term	Brief Meaning
Algorithm	A clear, repeatable, testable process for solving a problem
Design Phase	The planning and reasoning work before or alongside coding
Coding Phase	The implementation of the designed procedure
Functional Requirement	What the algorithm must produce or do
Non-functional Requirement	How well the algorithm should behave under constraints
Data-Intensive	Work shaped mainly by the amount, speed, or variety of data
Compute-Intensive	Work shaped mainly by the amount of processing required
Edge Case	A boundary or unusual input that tests hidden assumptions

8

Southwest
Tech

How To Use the Textbook

The textbook may look more mathematical than this course feels at first.

For now:

- ▶ skim for the idea
- ▶ notice key terms
- ▶ mark what looks unfamiliar
- ▶ do not panic over every formula
- ▶ connect the concept back to Python functions

9

Course Pattern

-  - Lecture concept
-  - Instructor demo
-  - Related but different lab
-  - Evidence and explanation
-  - Responsible AI use

10

Today's Question

How do we turn a vague instruction into a testable algorithm?

VAGUE INSTRUCTION



TESTABLE ALGORITHM

Condition	Action	Expected Result
Input is valid	Process the input	Output is calculated
Input is missing	Show error message	Error message is displayed
Input is invalid	Reject input	No output is produced
Input is out of range	Clamp to valid range	Adjusted value is used



11

What Counts As Success Today

-  name the inputs and outputs
-  state assumptions
-  write precise steps
-  test normal and edge cases
-  revise when evidence exposes ambiguity

12

Algorithm

Textbook Review

Southwest Tech

What is an Algorithm?

An algorithm is a structured process for solving a problem.

It should be:

- ▶ clear
- ▶ repeatable
- ▶ testable
- ▶ explainable
- ▶ implementable

ALGORITHMS
Design and Analysis

DATA STRUCTURES
& ALGORITHMS

DISCRETE MATHEMATICS

COMPUTER SCIENCE
FUNDAMENTALS

PROBLEM SOLVING
WITH ALGORITHMS

13

Algorithm

Textbook Review

Southwest Tech

Design and Coding

Algorithm work has two connected phases:

- ▶ Design: decide the logic
- ▶ Coding: implement the logic

Good coding cannot fully rescue *unclear* design.

ALGORITHMS
Design and Analysis

DATA STRUCTURES
& ALGORITHMS

DISCRETE MATHEMATICS

COMPUTER SCIENCE
FUNDAMENTALS

PROBLEM SOLVING
WITH ALGORITHMS

14

Algorithm

Textbook Review

Southwest Tech

Requirements

Functional requirements:

- ▶ What result should the algorithm produce?

Non-functional requirements:

- ▶ How well should it perform?
- ▶ How readable should it be?
- ▶ How well should it handle growth?

ALGORITHMS
Design and Analysis

DATA STRUCTURES
& ALGORITHMS

DISCRETE MATHEMATICS

COMPUTER SCIENCE
FUNDAMENTALS

PROBLEM SOLVING
WITH ALGORITHMS

15

Algorithm

Textbook Review

Southwest
Tech

Three Design Questions

When designing an algorithm, ask:

1. Does it produce the expected result?
2. Is this a reasonable way to get the result?
3. What happens when the data gets larger?



16

Southwest
Tech

Algorithmic Meaning

"Algorithm" names the reasoning pattern,
not only the final code.



17

Southwest
Tech

An Algorithm is A Procedure

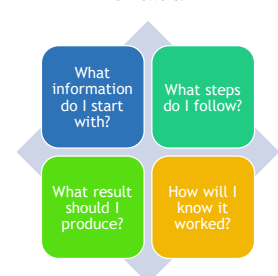
It Answers:

What information do I start with?

What steps do I follow?

What result should I produce?

How will I know it worked?



18

Same Idea, Different Forms

One algorithm. Many valid ways to express it.

Plain English	Pseudocode	Python Code	Table of Rules	Flowchart / Diagram												
Find the largest number in a list. 1. Start with the first number as the largest. 2. Compare the next number to the largest. 3. If the next number is larger, update the largest. 4. Repeat until there are no numbers left. 5. The largest number is the result.	<pre>function max(A): largest ← A[0] for each x in A[1:]: if x > largest: largest ← x return largest</pre>	<pre>def max_list(A): largest = A[0] for x in A[1:]: if x > largest: largest = x return largest</pre>	<table><thead><tr><th>Step</th><th>Action</th></tr></thead><tbody><tr><td>1</td><td>Set largest to the first number.</td></tr><tr><td>2</td><td>Look at the next number.</td></tr><tr><td>3</td><td>If it is larger than largest, update largest.</td></tr><tr><td>4</td><td>Repeat until there are no numbers left.</td></tr><tr><td>5</td><td>The largest number is the result.</td></tr></tbody></table>	Step	Action	1	Set largest to the first number.	2	Look at the next number.	3	If it is larger than largest, update largest.	4	Repeat until there are no numbers left.	5	The largest number is the result.	<pre>graph TD Start([Start]) --> SetLargest[Set largest = first number] SetLargest --> MoreNumbers{More numbers left?} MoreNumbers -- Yes --> ReadNext[Read next number] ReadNext --> IsLarger{Is next number > largest?} IsLarger -- Yes --> UpdateLargest[Update largest] UpdateLargest --> MoreNumbers IsLarger -- No --> ReturnLargest[Return largest] MoreNumbers -- No --> ReturnLargest ReturnLargest --> End([End]) End --> Split(()) Split --> Left[] Split --> Right[] style Left fill:none,stroke:none style Right fill:none,stroke:none</pre>
Step	Action															
1	Set largest to the first number.															
2	Look at the next number.															
3	If it is larger than largest, update largest.															
4	Repeat until there are no numbers left.															
5	The largest number is the result.															

19

“So, What Makes it Algorithmic?”

The process must be

- ▶ Ordered
- ▶ Precise
- ▶ Repeatable
- ▶ Testable
- ▶ Connected to a result

20

Vague Instructions Hide Decisions

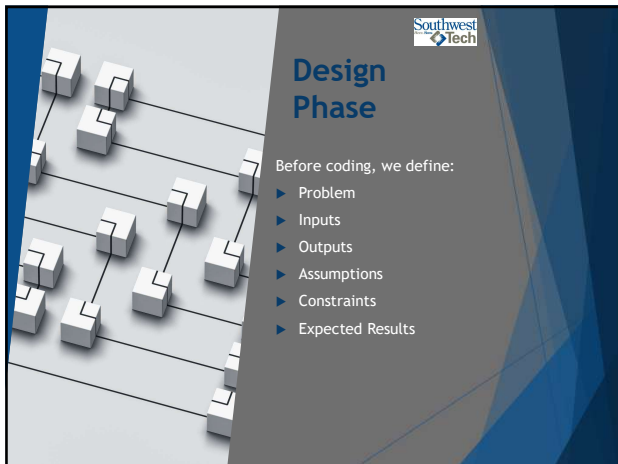
Vague Words:

- Low
- Soon
- Many
- Best
- Reasonable
- Urgent

21



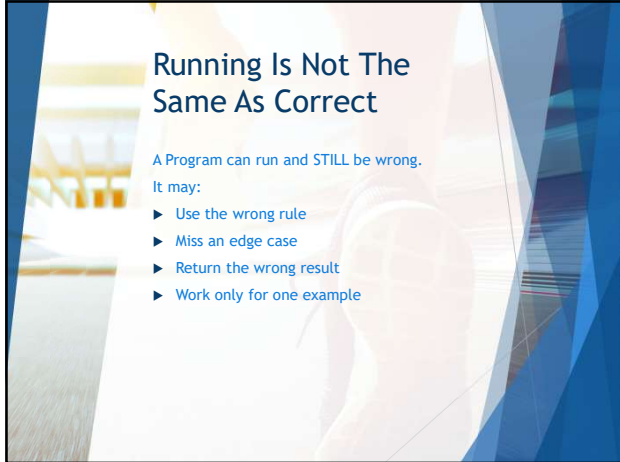
22



23



24

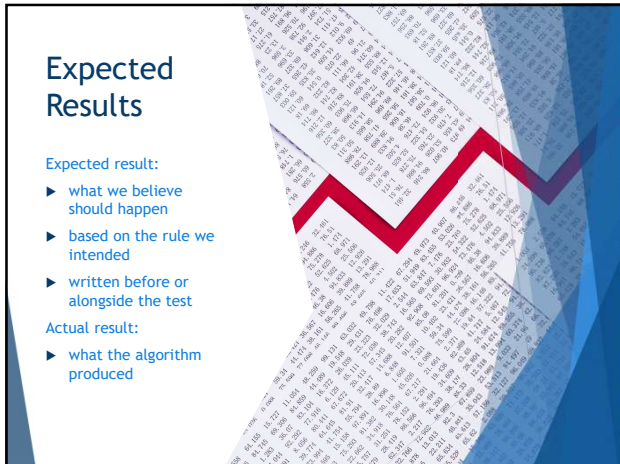


Running Is Not The Same As Correct

A Program can run and STILL be wrong.
It may:

- ▶ Use the wrong rule
- ▶ Miss an edge case
- ▶ Return the wrong result
- ▶ Work only for one example

25



Expected Results

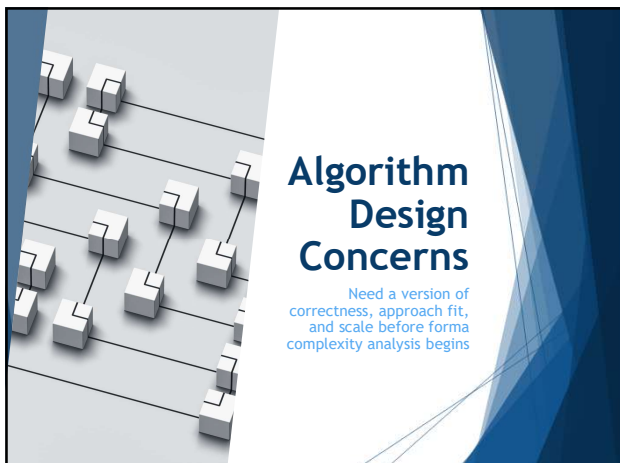
Expected result:

- ▶ what we believe should happen
- ▶ based on the rule we intended
- ▶ written before or alongside the test

Actual result:

- ▶ what the algorithm produced

26



Algorithm Design Concerns

Need a version of correctness, approach fit, and scale before formal complexity analysis begins

27



Concern 1: Expected Result

First question:
“Does the algorithm produce
the expected result?”

Evidence may include:

- ▶ test cases
- ▶ input/output tables
- ▶ traces
- ▶ before/after comparisons

28



Concern 2: Reasonable Approach

Second question:
“Is this a reasonable way to
get the result?”

Consider:

- ▶ rule order
- ▶ readability
- ▶ unnecessary steps
- ▶ confusing assumptions

29



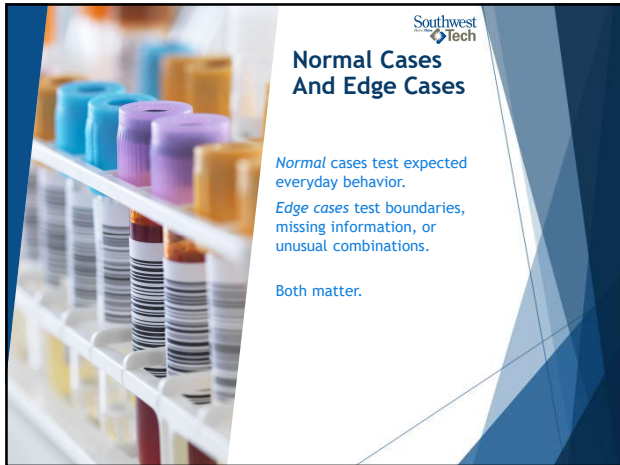
Concern 3: Larger Data

Third question:
“What happens when the
input gets larger?”

Ask:

- ▶ Does the work grow slowly?
- ▶ Does the work grow quickly?
- ▶ Does the algorithm repeat work?

30



Southwest
Tech

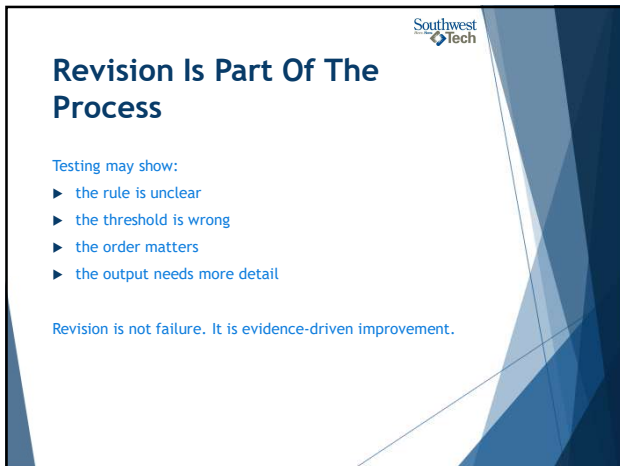
Normal Cases And Edge Cases

*Normal cases test expected
everyday behavior.*

*Edge cases test boundaries,
missing information, or
unusual combinations.*

Both matter.

31



Southwest
Tech

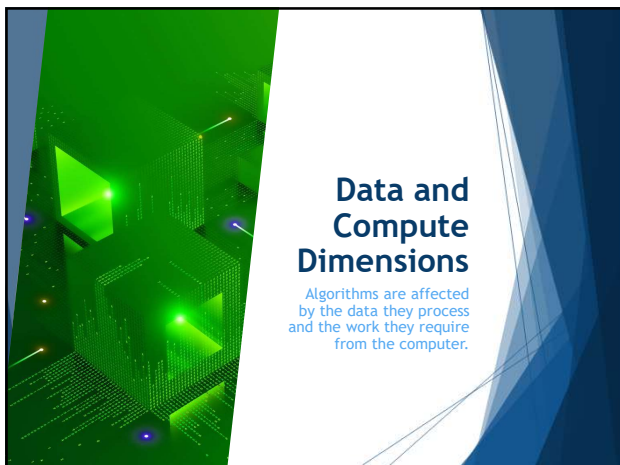
Revision Is Part Of The Process

Testing may show:

- ▶ the rule is unclear
- ▶ the threshold is wrong
- ▶ the order matters
- ▶ the output needs more detail

Revision is not failure. It is evidence-driven improvement.

32



Southwest
Tech

Data and Compute Dimensions

Algorithms are affected
by the data they process
and the work they require
from the computer.

33

Data-Intensive Algorithms

A data-intensive algorithm is shaped mainly by the data.

Questions:

- ▶ How much data?
- ▶ How fast does it arrive?
- ▶ How many forms does it take?

34

The Data Dimension

Data can be described by:

- ▶ volume: how much
- ▶ velocity: how fast
- ▶ variety: how many forms

These affect which algorithmic approach fits.

35

Compute-Intensive Algorithms

A compute-intensive algorithm is shaped mainly by how much processing it requires.

Examples may include:

- ▶ simulations
- ▶ image processing
- ▶ cryptography
- ▶ model training
- ▶ repeated calculations

36



Both Data And Compute

Some algorithms are affected by both:

- ▶ large data volume
- ▶ frequent updates
- ▶ varied data types
- ▶ heavy processing

Design must consider both dimensions.

37



Week 1: Practical Takeaway

Before coding, ask:

- ▶ What data do I need?
- ▶ How much data might there be?
- ▶ What work must the algorithm do?
- ▶ How will I test the result?



38

Demo Scenario

Demo problem:
"Should a student bring a laptop charger to campus?"

Inputs:

- ▶ battery percent
- ▶ expected hours on campus
- ▶ outlet access

Output:

- ▶ bring charger
- ▶ charger optional



39

Southwest
Tech

Vague Rule Version

Initial rule:

- ▶ If the battery is low, bring the charger.
- ▶ If campus time is long and outlets are unreliable, bring the charger.
- ▶ Otherwise, the charger is optional.



40

Southwest
Tech

Precise Rule Version

Revised rule:

- ▶ If battery percent is 40 or below, bring the charger.
- ▶ Else if expected hours is 4 or more and outlet access is false, bring the charger.
- ▶ Otherwise, the charger is optional.



41

Southwest
Tech

Representation Bridge

Same algorithm, different forms:

- ▶ precise plain English
- ▶ pseudocode
- ▶ Python-style logic
- ▶ test table



42



Demo Evidence



Watch for:

- ▶ expected result
- ▶ actual result
- ▶ pass or fail
- ▶ boundary case
- ▶ revised rule

43



Demo Add-On: Console Color



After the plain demo works, add light console color.

The color helps highlight:

- ▶ section headings
- ▶ passed tests
- ▶ failed tests
- ▶ important boundary cases

44



Add-On Principle: Presentation Is Separate From Logic



Keep these separate:

- ▶ algorithm *logic*: makes the decision
- ▶ evidence *output*: shows what happened
- ▶ presentation *layer*: makes output easier to scan

45



From Demo to Lab

In the lab, you will choose a different scenario and create:

- ▶ problem statement
- ▶ inputs and outputs
- ▶ assumptions
- ▶ pseudocode or Python
- ▶ normal and edge-case tests
- ▶ revision note

46



Walkthrough Support

The walkthrough artifact can help you see the pattern.

If you use it, explain:

- ▶ what pattern it showed
- ▶ how you adapted it
- ▶ what you still decided yourself

47



README Evidence



Your README should show:

- ▶ inputs and outputs
- ▶ assumptions
- ▶ test table
- ▶ expected vs actual results
- ▶ before/after revision
- ▶ AI-use note, if applicable

48



49

Southwest Tech

Example: Expected Result Is Not Always Obvious



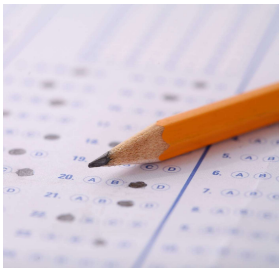
Two people may disagree about the expected result when:

- ▶ the rule is vague
- ▶ the threshold is missing
- ▶ priorities conflict
- ▶ information is incomplete

50

Southwest Tech

Example: Rule Order Matters



Changing rule order can change the result.

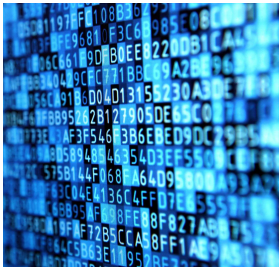
Ask:

- ▶ Which rule should be checked first?
- ▶ Which rule wins if two rules conflict?
- ▶ Should the algorithm stop early?

51

Southwest
Tech

Example: Small Data Can Hide Problems



An approach may seem fine with 5 examples.

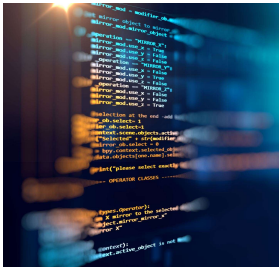
It may struggle with:

- ▶ 500 records
- ▶ 5,000 records
- ▶ repeated updates
- ▶ mixed data formats

52

Southwest
Tech

Example: Data And Compute Together



Recommendation systems can involve:

- ▶ many users
- ▶ many resources
- ▶ many tags or features
- ▶ frequent score updates
- ▶ repeated ranking calculations

53

Southwest
Tech

Wrap up



54



What To Carry Forward



Algorithms are not just code.
They are:

- ▶ designed
- ▶ represented
- ▶ tested
- ▶ revised
- ▶ explained

55



Next Step



For Lab 01:

- choose a scenario
- write precise rules
- create normal and edge-case tests
- compare expected and actual results
- revise one ambiguity

56



How To Use The Textbook For Next Week's Reading



For next week, read for growth behavior.

Focus on:

- ▶ what time complexity means
- ▶ what space complexity means
- ▶ why input size matters
- ▶ what $O(1)$, $O(n)$, $O(n^2)$, and $O(\log n)$ are trying to describe
- ▶ how best, worst, and average cases differ

Do not panic over formulas. Mark them, skim for meaning, and connect them back to the code examples.

57



58
